

|                                                                                                                                                                |                                                 |                            |                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|----------------------------|----------------------------------|
|  <b>INSTITUTO FEDERAL</b><br>Espírito Santo<br>Campus Cachoeiro de Itapemirim | <b>IFES - Campus Cachoeiro de Itapemirim-ES</b> |                            |                                  |
|                                                                                                                                                                | <b>Sistemas de Informação</b>                   |                            | <b>Disciplina:</b> Programação 1 |
|                                                                                                                                                                | <b>Professor:</b> Flávio Izo                    |                            | <b>Lista 10</b>                  |
| <b>Aluno:</b>                                                                                                                                                  |                                                 | <b>Conteúdo:</b> Algoritmo |                                  |

## LISTA DE DESAFIOS 01

### Elaboração de algoritmos

Só tente fazer o desafio se você já estiver feito os outros exercícios.

1) O Problema das Rainhas. Faça um algoritmo que consiga posicionar 4 rainhas em um tabuleiro de xadrez de 4x4 sem que uma fique na mesma posição da outra. Inicialmente você deve criar uma matriz 4x4. Você deve informar, inicialmente, as posições em que existir uma rainha atribuindo o valor “X”.

Por fim, teste para verificar se uma rainha está na mesma direção da outra. Acesse o link <https://flavioizo.com/algoritmos/NRainhas/> para efetuar eventuais testes.

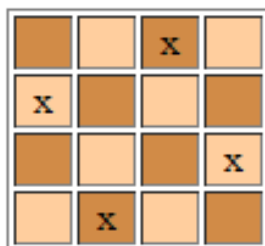


Figura 1: Exemplo de posicionamento de 4 rainhas

|                                                                                                                                                                |                                                 |                            |                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|----------------------------|----------------------------------|
|  <b>INSTITUTO FEDERAL</b><br>Espírito Santo<br>Campus Cachoeiro de Itapemirim | <b>IFES - Campus Cachoeiro de Itapemirim-ES</b> |                            |                                  |
|                                                                                                                                                                | <b>Sistemas de Informação</b>                   |                            | <b>Disciplina:</b> Programação 1 |
|                                                                                                                                                                | <b>Professor:</b> Flávio Izo                    |                            | <b>Lista 10</b>                  |
| <b>Aluno:</b>                                                                                                                                                  |                                                 | <b>Conteúdo:</b> Algoritmo |                                  |

2) Crie um algoritmo que leia uma matriz 10x10. Cada coluna se refere a um vetor coluna. Logo em seguida, estabeleça a distância euclidiana entre um vetor coluna e os demais. Assim, separe a menor distância e armazene em um vetor **RAST** de 10 posições. Por fim, informe a maior distância encontrada entre as menores distâncias entre as colunas.

Ex.:

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 5 | 2 | 3 | 4 | 3 | 2 | 2 | 4 |
| 1 | 2 | 3 | 6 | 3 | 9 | 9 | 4 | 3 | 3 | 9 |
| 2 | 3 | 9 | 7 | 9 | 8 | 1 | 9 | 9 | 4 | 1 |
| 3 | 4 | 8 | 3 | 8 | 7 | 3 | 1 | 8 | 5 | 3 |
| 4 | 5 | 7 | 4 | 7 | 2 | 5 | 3 | 7 | 6 | 5 |
| 5 | 6 | 2 | 9 | 6 | 4 | 5 | 5 | 2 | 7 | 5 |
| 6 | 7 | 4 | 1 | 5 | 5 | 8 | 6 | 5 | 8 | 5 |
| 7 | 8 | 5 | 3 | 8 | 4 | 7 | 2 | 8 | 3 | 8 |
| 8 | 9 | 4 | 5 | 7 | 3 | 6 | 1 | 7 | 2 | 7 |
| 9 | 0 | 3 | 6 | 6 | 1 | 2 | 1 | 6 | 4 | 6 |

Supondo a seguinte matriz acima, deve-se calcular a distância euclidiana entre a coluna 0 e a coluna 1, depois entre a coluna 0 e coluna 2 e assim até a coluna 0 e coluna 9. Então deve-se separar a menor distância encontrada e colocar na posição 0 do vetor **RAST**. Assim, deve-se calcular agora a distância euclidiana entre a coluna 1 e a coluna 0, depois entre a coluna 1 e coluna 2 e assim até a coluna 1 e coluna 9. Então deve-se separar a menor distância encontrada e colocar na posição 1 do vetor **RAST**. Todo esse processo deve ser feito até conseguir calcular as 10 menores distância e preencher todo o vetor **RAST**. Por fim, deve-se informar a maior distância que está preenchida no vetor **RAST**.

|                                                                                                                                                                |                                                 |                            |                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|----------------------------|----------------------------------|
|  <b>INSTITUTO FEDERAL</b><br>Espírito Santo<br>Campus Cachoeiro de Itapemirim | <b>IFES - Campus Cachoeiro de Itapemirim-ES</b> |                            |                                  |
|                                                                                                                                                                | <b>Sistemas de Informação</b>                   |                            | <b>Disciplina:</b> Programação 1 |
|                                                                                                                                                                | <b>Professor:</b> Flávio Izo                    |                            | <b>Lista 10</b>                  |
| <b>Aluno:</b>                                                                                                                                                  |                                                 | <b>Conteúdo:</b> Algoritmo |                                  |

### 3) Jogo da velha:

Crie um algoritmo em linguagem C para dois usuários jogarem o jogo da velha tradicional. Abaixo foi colocada uma tela de exemplo.

```

X | O | X
O | O | X
O | X |

--> Jogador 1
Linha: 3
Coluna: 3
Jogo encerrado. Jogador 1 venceu !

X | O | X
O | O | X
O | X | X

```

Figura 02: Tela de exemplo do jogo

### 4) Jogo de Craps: Faça um programa que implemente um jogo de Craps modelo “Pass Line”

#### Regras do jogo:

- O jogador lança um par de dados, obtendo um valor entre 2 e 12.
- Se, na primeira jogada, você tirar 7 ou 11, você é um "natural" e ganhou.
- Se você tirar 2, 3 ou 12 na primeira jogada, isto é chamado de "craps" e você perdeu.
- Se, na primeira jogada, você fez um 4, 5, 6, 8, 9 ou 10, este é seu "Ponto". Seu objetivo agora é continuar jogando os dados até tirar este número novamente. Você perde, no entanto, se tirar um 7 antes de tirar este Ponto novamente.

### 5) Jogo da Forca:

Faça um jogo da forca em linguagem C. Esse jogo deve possuir as seguintes regras:

- Mostrar na tela uma dica para a palavra a ser descoberta
- Mostrar quais letras já foram informadas
- Mostrar quantos erros e acertos o usuário já teve
- Mostrar a palavra sendo preenchida
- Mostrar tentativas restantes
- Mostrar o “boneco” sendo preenchido (esse não é obrigatório)

*OBS.: A elaboração desses exercícios é meramente para fins educacionais de aprendizagem de lógica de programação e não tem o objetivo de torna-lo um jogador assíduo.*

Bom exercício! ☺